Creator Playbook

HACKERABAD

Creator Playbook

Section 1: Introduction....2
Section 2: Tools You Need....2
Section 3: Collaborate with Community via Discord....3
Section 4: How to Create a Game with Phaser....3
1. Ideation....3
2. Development....4
3. Polish....5
4. Deployment....6

Section 1: Introduction

Welcome to Hackerabad Game Dev Session

Game development is the process of creating interactive experiences using:

- Code
- Visuals (art)
- Sound
- Logic

In this session, you'll learn how to build a real playable game using Phaser.js, a powerful JavaScript game framework.

What You'll Build:
A clickable interactive game
With:

- Score system
- Sound effects
- Animations

Mindset:
You don't need to know everything.
You just need to build → test → improve.

Section 2: Tools You Need

Before the session, make sure you have:

-Code Editor
VS Code (recommended)

-Browser
Chrome / Edge

-Basic Setup
A folder for your game project

-AI Tools
We encourage AI-assisted coding (vibecoding):
Use:
ChatGPT
GitHub Copilot
Cursor (optional)

Section 3: Collaborate with Community via Discord

Discord is our primary platform for communication, feedback, and event updates. You're welcome to join our **community channel**!

Engage with fellow community members, share your game projects, and receive expert feedback to transform your game from a rough concept into a polished, industry-standard product all in the **Main Chat**.

Section 4: How to Create a Game with Phaser

This section is your **core workflow** for building any game during this session.

We break game development into **4 simple steps**:

Ideation → Development → Polish → Deployment

1. Ideation

Every game starts with an idea — but you don't need a perfect one.

What You Do:

- Think of a simple game idea

Examples:

- Click object → increase score
- Hit character → play animation
- Avoid obstacles

Use AI to Expand Your Idea

Once you have a basic idea, ask AI:

""I want to build a "click the character to score points" game using Phaser.

Give me:

- core mechanics
- features to add
- simple structure""

2. Development

This is where your game comes to life.

Step 1: Generate Base Code Using AI

Ask AI:

Generate a basic Phaser game using:

- preload()
- create()
- update()

Include:

- one object in center
- click interaction
- score system

Important:

- Use the **code structure from this playbook**
- Don't generate random messy code

Step 2: Follow the Core Flow

Now you'll work on top of that code:

1. Load Assets

```
this.load.image('object', 'assets/object.png');
```

2. Add Objects

```
this.object = this.add.image(400, 300, 'object');
```

3. Add Interaction

```
this.object.setInteractive();

this.object.on('pointerdown', () => {

  this.handleClick();

});
```

4. Add Logic

```
handleClick() {
  this.score++;
}
```

Goal of This Step:

- Get a **working playable game**
- Even if it's very basic

Don't aim for perfection — aim for "it works"

3. Polish

Now we make the game **fun and satisfying**

Add Sound Effects

- Click sound
- Hit sound

```
this.sound.play('click');
```

Add Animations

Example:

```
this.tweens.add({
  targets: this.object,
  scale: 1.2,
  duration: 100,
  yoyo: true
});
```

Goal of This Step:

Make your game feel **alive**

4. Deployment

Now let's make your game playable.

Run Your Game

- Open `index.html` in your browser
- Or use Live Server (VS Code)

Test Your Game

Check:

- Does clicking work?
- Is the score updating?
- Are sounds playing?

Improve Your Game

Ask AI:

How can I improve this Phaser game to make it more fun?

Try Adding:

- More objects
- Difficulty increase
- Better animations

Goal of This Step:

- Turn your game from **basic** → **better**

How to?

Game Development Guide - Phaser (Hackerabad Edition)

Overview

This comprehensive guide provides developers with the essential knowledge and code snippets needed to create games using Phaser.js framework.

Coding Guidelines

Core Development Principles

Guideline	Directive
1. Code Structure	Follow this order in every game: constructor for variables → Custom functions → preload → create → update → end
2. Base Code	Use the provided base code without editing — it's required for the game to run smoothly
3. Variable Initialization	Always initialize your variables inside the constructor of the game scene
4. Asset Loading	Load all images, animation, audio, and fonts in the preload() function only
5. Create Method Usage	Use the create() function only after preload to set up your game elements
6. Object Declaration	Add your game-specific objects (like enemies, buttons) between update() and updateScore() blocks
7. Predefined Logic	Trigger logic like pause, restart, game over — but don't edit the actual functions
8. Use Class Instead of Standalone	Write your game scene using a class — avoid writing separate standalone functions outside it

Base Template - File has been shared

Mobile Compatibility(Advanced)

Orientation Detection

```
// Portrait detection
const isPortrait = this.game.config.height > this.game.config.width;

// Scale factor for automatic resizing
const scaleFactor = isPortrait ? this.game.config.width / 800 : this.game.config.width /
1200;

// Example usage
const tube = this.add.sprite(
    tubeSpacing * (index + 1),
    isPortrait ? this.game.config.height * 0.6 : this.game.config.height * 0.7,
    'tube'
).setScale(0.7 * scaleFactor).setDepth(10);
```

Orientation Helper Functions

```
getOrientation() {
    return window.innerWidth > window.innerHeight ? 'landscape' : 'portrait';
}

getOrientationSizes() {
    const orientation = this.getOrientation();
    return _CONFIG.deviceOrientationSizes[orientation];
}
```

Mobile Detection

```
this.isMobile = !this.sys.game.device.os.desktop;
this.currentOrientation = this.getOrientation();
```

Responsive Text

```
this.timerText = this.add.bitmapText(
    this.game.config.width / 2,
    isPortrait ? 80 * scaleFactor : 50 * scaleFactor,
```

```
    'pixelfont',
    `Time: ${this.timeLeft}`,
    32 * scaleFactor
).setOrigin(0.5);
```

VFX Integration

Overview

Enhance your game's visual appeal using the VFX Library provided by Braincade SDK. The library includes reusable animation and effect utilities.

Library Source:
`https://raw.githubusercontent.com/ks-braincade/braincadeSDK/main/src/vfxLibrary.js`

Integration Steps

1. Initialize VFX in Constructor

```
constructor() {
    super({ key: 'GameScene' });
    this.vfx = new VFXLibrary(this);
}
```

2. Use VFX Functions

```
// Rotate player 360 degrees in 500ms
this.vfx.rotateGameObject(this.player, 500, 360);

// Blink enemy 4 times quickly
this.vfx.blinkEffect(this.enemy, 300, 4);

// Scale coin up for a brief moment
this.vfx.scaleGameObject(this.coin, 1.5, 200);
```

3. Common VFX Use Cases

- **Hit feedback:** `shakeGameObject()`
- **Collectibles:** `shineEffect()`
- **Movement:** `trailEffect()`

Animation Integration

Using Spritesheets for Frame-by-Frame Animation

Please go through this Gemini Chat to see an example on how to add animations.

Step 1: Prepare Animation Frames

- Use individual frame images with consistent dimensions (e.g., 16×16)
- Export frames from master spritesheet using tools like Spritesheet Cutter or this

Step 2: Pack the Spritesheet

- Use CodeAndWeb Sprite Sheet Packer
- Export both `.png` spritesheet and corresponding `.json` file
- Upload spritesheet to asset list
- Host JSON file using reliable uploader (e.g. catbox.moe)

Step 3: Load Spritesheet in preload()

```
this.load.atlas('fighterAtlas', _CONFIG.imageLoader['spritesheet'],
'https://files.catbox.moe/x53w7u.json');
```

Step 4: Define Animation in create()

```
this.anims.create({
  key: 'walkRight',
  frames: [
    { key: 'fighterAtlas', frame: 'image_102-removebg-preview.png' },
    { key: 'fighterAtlas', frame: 'image_103-removebg-preview.png' },
    { key: 'fighterAtlas', frame: 'image_103-removebg-preview(1).png' },
    { key: 'fighterAtlas', frame: 'image_104-removebg-preview.png' },
    { key: 'fighterAtlas', frame: 'image_105-removebg-preview.png' }
  ],
  frameRate: this.isMobile ? 8 : 10,
  repeat: -1
});
```

Step 5: Play Animation

```
this.player.anims.play('walkRight', true);
```

Important Notes

- Always use consistent frame dimensions
- Mismatched sizes will break collider boundaries
- Host JSON files on stable services
- Break large sprite sheets into frames first, then repack

Sound Integration

Step 1: Create or Source Sounds

- Use AI tools or sound creation software
- Upload sounds to platforms like whyp.it
- Copy generated URLs

Step 2: Configure Sound URLs

Add URLs to the `soundsLoader` dictionary in your configuration:

```
"soundsLoader": {
  "background": "https://example.com/background-music.mp3",
  "jump": "https://example.com/jump-sound.wav",
  "damage": "https://example.com/damage-sound.wav",
  "lose": "https://example.com/lose-sound.mp3",
  "countdown": "https://example.com/countdown-sound.mp3"
}
```

Step 3: Load Sounds Dynamically

```
preload() {
  for (const key in _CONFIG.soundsLoader) {
    this.load.audio(key, [_CONFIG.soundsLoader[key]]);
  }
}
```

Step 4: Use Sounds in Game

```
create() {
  // Play background music
  this.sound.play('background', { loop: true, volume: 0.5 });
```

```
}

onPlayerJump() {
   this.sound.play('jump');
}

onPlayerDamage() {
   this.sound.play('damage');
}

onGameLose() {
   this.sound.play('lose');
}

playCountdown() {
   this.sound.play('countdown');
}
```

Step 5: Configure Sound Settings

```
// Looped background music
this.sound.play('background', { volume: 0.5, loop: true });

// Single jump sound
this.sound.play('jump', { volume: 1.0 });
```

CODE ITERATION:

Code Iteration Workflow

1. **Upload the Game File:**
   Begin by uploading the `game.js` file to the AI platform.

2. **Enter a Precise Prompt:**
   Use a clear and specific prompt to request modifications. For example:
   "Read the code and make [specific changes required for the game]. Do not rewrite the entire code—only provide the necessary code blocks or modifications."

3. **Apply AI-Generated Changes:**
   Use the code blocks provided by the AI to replace or update the corresponding sections in the existing `game.js` file.

4. **Add Required Assets:**
   4.1. Upload any new assets needed for the game directly through the chat window.
   4.2. You can iterate existing assets using the chat window and write a sample prompt after you drag the assets to the chat window, For e.g., “Change this asset to ____”.

5. **Update Asset References in `config.json`:**
   In the `config.json` file, under the `imageLoader` key, rename any AI-generated asset names to meaningful and relevant identifiers.
   Example:
   Rename `"extra_grandpa_max_2025-07-15 16:57:05.823682"` to `"enemy"`.

6. **Repeat as Needed:**
   For additional updates or new features, repeat the steps above in a continuous loop of improvement.

Example prompts to generate a spritesheet through chatgpt, gemini, or any other AI:

"Create a detailed pixel art frame animation for a game, where the final image is divided into multiple sub-images, each serving as a continuous animation keyframe. Design the sequence to depict **[a wizard casting a spell: begin with intricate hand motions, then show the wizard conjuring a vibrant fireball, and finally capture the moment of casting the fireball.]** Ensure the keyframes transition smoothly and continuously, and include as many frames as possible to achieve a high level of fluidity and detail in the animation. It should be side view in 2d pixelated with 16 x 16 size of 5 frames each"

Replace the [ ] part with the character you want and the animation style

You can generate spritesheet using these tools -> Resources

Best Practices

1. **Code Organization**: Always follow the prescribed structure order
2. **Asset Management**: Load all assets in preload() only

3. **Mobile Optimization**: Use responsive scaling and orientation detection
4. **Performance**: Use VFX library for consistent visual effects
5. **Sound Design**: Implement dynamic sound loading for modularity
6. **Animation**: Maintain consistent frame dimensions for smooth animations

Troubleshooting

- **Animation Issues**: Ensure frame names match exactly with JSON file
- **Sound Problems**: Verify URL accessibility and file format compatibility
- **Mobile Rendering**: Check orientation detection and scaling factors
- **VFX Performance**: Combine effects thoughtfully to avoid overload

---

This guide provides the foundation for developing games on the Aicade platform. Follow these guidelines to ensure compatibility and optimal performance across all devices.

Resources

Coding

Basics of Phase Game Development

1. Official Phaser tutorial to create games
2. Detailed Phaser learning resource playlist
3. Learn Phaser for beginners - useful to get familiar with
4. Phaser Crash course for devs who prefer other languages
5. Phaser Code Examples on Phaser's official Github
6. More game code examples for Phaser by the community
7. Some example base games to learn Phaser coding

AI tools for Coding

Complete Game Code Generation

1. Blackbox AI
2. DeepAI
3. Claude by Anthropic

Snippet or Feature Generation

1. ChatGPT by OpenAI Best for sprite animations, physics implementations, or event handlers.
2. GitHub Copilot Provides on-the-fly suggestions for Phaser.js
3. Cursor Great for iterating on small portions of Phaser.js code

Visual Generation Tool:

1. Recraft.ai - Perfect for characters or scenes.
2. Layer.ai - World-building visuals, including detailed maps, and Icons/UI.
3. Scenario.ai - High-quality game assets of all types, from characters to props.
4. DALL-E - Low effort to prompt, making it beginner-friendly.
5. MidJourney - Detailed visuals but requires learning advanced prompt techniques
6. Getimg.ai - Offers multiple models to choose from

Animation:

Guide: Best Places to Obtain or Generate Spritesheets

1. The Spriters Resource - For those who want retro or recognizable assets from existing games.
2. CraftPix - For professional assets for commercial projects.
3. Aseprite - Developers who want to create unique, personalized assets.
4. Itch.io - Developers looking for unique or niche assets
5. Freepik & GameArt2D - Developers looking for quick, ready-to-use assets
6. GODMODE - Create spritesheet from custom images
7. LPC GENERATOR - Create a custom character through this spritesheet generator
8. Tools 3ox - Sprite Sheet Cutter
9. ezgif - Sprite Sheet Cutter

Sound:

Sound creation libraries or AI sound creation:

- Libraries
  Pixabay:- https://pixabay.com/sound-effects/
  Freesound:- https://freesound.org/
  ZapSplat:- https://www.zapsplat.com/
  Mixkit:- https://mixkit.co/free-sound-effects/
  Youtube:- https://www.youtube.com/@gamingsoundfx/videos
  Quick Sounds:- https://quicksounds.com/

- AI tool:-
  beethoven.ai:- https://www.beatoven.ai
  AudioCipher:- https://audiocipher.com
  Suno AI:- https://suno.com
  Udio:- https://udio.com
  SoundGen:- https://soundgen.io
  Riffusion:- https://riffusion.com
  Mubert:- https://mubert.com
  Google MusicFX (formerly MusicLM):- https://google.com/musicfx
  HookPad Aria:- https://hooktheory.com
  Lemonaide:- https://lemonaide.io
  AIVA:- https://aiva.ai

BandLab SongStarter:- https://bandlab.com
Audacity OpenVINO AI Effects:- https://audacityteam.org
Samplab:- https://samplab.com
Spotify Basic Pitch:- https://basicpitch.spotify.com
RipX DAW Pro:- https://hitnmix.com/ripx
Melodyne:- https://celemony.com
Soundraw:- https://soundraw.io
Boomy:- https://boomy.com
Soundful:- https://soundful.com
MuseNet (OpenAI):- https://openai.com/musenet